

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 4”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan praktikum dan laporan ini dengan baik. Laporan praktikum ini disusun sebagai dokumentasi dari praktikum pekan ke-4 mata kuliah Struktur Data mengenai konsep **Queue** (antrian) beserta berbagai implementasinya menggunakan bahasa pemrograman Java. Melalui praktikum ini, kami mempelajari dua pendekatan utama dalam mengimplementasikan Queue, yaitu menggunakan struktur data bawaan Java (LinkedList) dan implementasi manual menggunakan array sirkular (Circular Array). Selain itu, kami juga mempelajari penggunaan Iterator, serta aplikasi Queue untuk membalik urutan data (reverse).

Kami mengucapkan terima kasih yang sebesar-besarnya kepada dosen pengampu, Bapak Dr. Wahyudi, S.T., M.T., serta asisten praktikum Sherly Sukmadira yang telah memberikan bimbingan, arahan, serta materi praktikum yang jelas. Terima kasih juga kami sampaikan kepada rekan-rekan mahasiswa yang telah saling membantu selama sesi praktikum berlangsung. Praktikum ini sangat membantu dalam memperdalam pemahaman mengenai struktur data linier, khususnya konsep First In First Out (FIFO) dan teknik enkapsulasi operasi antrian. Kami menyadari bahwa laporan ini masih memiliki kekurangan dan jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan laporan di masa mendatang.

Padang, 30 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Iterasi Queue dengan Iterator (IterasiQueue).....	2
2.1.1 Teori	2
2.1.2 Langkah-langkah	2
2.1.3 Contoh Kode	2
2.1.4. Hasil	3
2.1.5 Analisis.....	3
2.2 Implementasi Queue dengan Array (QueueArray)	3
2.2.1 Teori	3
2.2.2 Langkah-langkah	3
2.2.3 Contoh Kode	4
2.2.5 Analisis	4
2.3 Driver Queue Array (QueueArrayDriver).....	4
2.2.1 Teori	4
2.2.2 Langkah-langkah	4

2.2.3 Contoh Kode	5
2.2.4 Hasil	5
2.2.5 Analisis	5
2.4 Queue dengan LinkedList (QueueLinkedList)	5
2.2.1 Teori	5
2.2.2 Langkah-langkah	5
2.2.3 Contoh Kode	6
2.2.4 Hasil	6
2.2.5 Analisis	6
2.5 Aplikasi Queue – Reverse Data (ReverseData)	6
2.2.1 Teori	6
2.2.2 Langkah-langkah	6
2.2.3 Contoh Kode	7
2.2.4 Hasil	7
2.2.5 Analisis	7
2.6 Tugas Pekan 2	8
2.5.1. Analisis	8
BAB III KESIMPULAN	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Queue atau antrian merupakan salah satu struktur data linier yang sangat penting dalam ilmu komputer. Konsep Queue mengikuti prinsip **First In First Out (FIFO)**, di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Struktur ini banyak diterapkan dalam sistem antrian seperti antrian printer, antrian proses pada sistem operasi, simulasi lalu lintas, dan buffer pada jaringan komputer.

Pada praktikum kali ini, kami mempelajari berbagai cara mengimplementasikan Queue di Java, mulai dari penggunaan kelas siap pakai `LinkedList` yang mengimplementasikan interface `Queue`, hingga implementasi manual menggunakan array dengan teknik circular queue untuk menghemat ruang. Selain itu, kami juga mempraktikkan penggunaan Iterator untuk traversal dan penerapan Queue untuk membalik urutan data.

1.2 Tujuan

1. Memahami konsep dasar struktur data **Queue** dan prinsip FIFO.
2. Mengimplementasikan Queue menggunakan `LinkedList` dari Java `Collection Framework`.
3. Mengimplementasikan Queue secara manual menggunakan **Circular Array**.
4. Memahami dan menggunakan Iterator untuk mengakses elemen Queue secara berurutan.
5. Menerapkan Queue untuk menyelesaikan masalah sederhana seperti membalik urutan data (reverse).

1.3 Manfaat

1. Mahasiswa dapat memahami perbedaan antara implementasi Queue menggunakan struktur bawaan dan implementasi manual.
2. Meningkatkan kemampuan dalam merancang dan menganalisis efisiensi operasi enqueue dan dequeue.
3. Memberikan fondasi yang kuat untuk memahami aplikasi Queue pada permasalahan yang lebih kompleks di bidang Struktur Data dan Algoritma.

BAB II

PEMBAHASAN

2.1 Iterasi Queue dengan Iterator (IterasiQueue)

2.1.1 Teori

Iterator merupakan interface pada Java Collection Framework yang digunakan untuk melakukan iterasi (penelusuran) elemen-elemen dalam koleksi secara berurutan tanpa mengetahui struktur internalnya. Pada Queue, iterator memungkinkan kita menampilkan seluruh elemen tanpa menghapusnya.

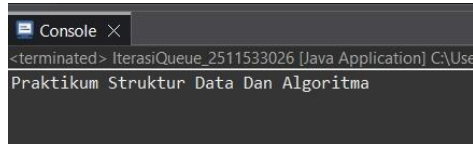
2.1.2 Langkah-langkah

1. Membuat objek Queue<String> menggunakan LinkedList.
2. Menambahkan beberapa elemen ke dalam Queue.
3. Mendapatkan Iterator dari Queue tersebut.
4. Menggunakan perulangan while dengan hasNext() dan next() untuk mencetak elemen.

2.1.3 Contoh Kode

```
1 package Pekan4_2511533026;
2
3 import java.util.Iterator;
4
5 public class IterasiQueue_2511533026 {
6     public static void main (String args[]) {
7         Queue<String> q_3026 = new LinkedList<>();
8
9         q_3026.add("Praktikum");
10        q_3026.add("Struktur");
11        q_3026.add("Data");
12        q_3026.add("Dan");
13        q_3026.add("Algoritma");
14        Iterator<String> iterator = q_3026.iterator();
15        while (iterator.hasNext()) {
16            System.out.print(iterator.next() + " ");
17        }
18    }
19 }
```

2.1.4 Hasil



2.1.5 Analisis

Penggunaan Iterator sangat aman dan fleksibel karena tidak mengubah struktur Queue. Output yang dihasilkan menunjukkan elemen tetap dalam urutan FIFO.

2.2 Implementasi Queue dengan Array (QueueArray)

2.2.1 Teori

Queue dapat diimplementasikan menggunakan array dengan teknik Circular Queue (array sirkular) untuk menghindari pemborosan ruang setelah operasi dequeue berulang. Variabel front, rear, dan size digunakan untuk mengelola posisi antrian.

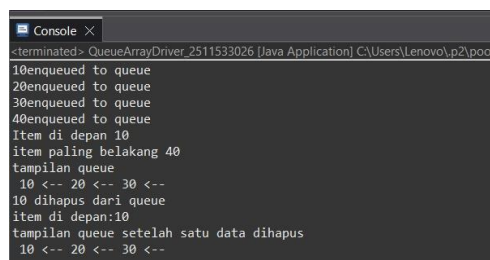
2.2.2 Langkah-langkah

1. Membuat kelas QueueArray dengan konstruktor yang menerima kapasitas.
2. Mengimplementasikan method isFull(), isEmpty(), enqueue(), dequeue(), front(), rear(), dan display().
3. Menggunakan operasi modulo (%) untuk membuat array bersifat circular.

2.3.3 Contoh Kode

```
1 package Pekan4_2511533026;
2
3 public class QueueArrayDriver_2511533026 {
4     public static void main (String[] args) {
5         QueueArray_2511533026 queue = new QueueArray_2511533026(1000);
6         queue.enqueue_3026(10);
7         queue.enqueue_3026(20);
8         queue.enqueue_3026(30);
9         queue.enqueue_3026(40);
10        System.out.println("Item di depan " + queue.front_3026());
11        System.out.println("item paling belakang " + queue.rear_3026());
12        System.out.println("tampilan queue");
13        queue.display_3026();
14        System.out.println();
15        System.out.println(queue.dequeue_3026() + " dihapus dari queue");
16        System.out.println("item di depan:" + queue.front_3026());
17        System.out.println("tampilan queue setelah satu data dihapus");
18        queue.display_3026();
19    }
20 }
21
```

2.3.4 Hasil



```
<terminated> QueueArrayDriver_2511533026 [Java Application] C:\Users\Lenovo\p2\poo
10enqueued to queue
20enqueued to queue
30enqueued to queue
40enqueued to queue
Item di depan 10
item paling belakang 40
tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
item di depan:10
tampilan queue setelah satu data dihapus
10 <-- 20 <-- 30 <--
```

2.3.5 Analisis

Driver berhasil menguji seluruh method dengan benar. Hasil menunjukkan perilaku FIFO yang konsisten.

2.4 Queue dengan LinkedList (QueueLinkedList)

2.4.1 Teori

LinkedList di Java sudah mengimplementasikan interface Queue, sehingga sangat mudah digunakan untuk merepresentasikan antrian dengan ukuran dinamis.

2.4.2 Langkah-langkah

1. Membuat Queue<Integer> menggunakan LinkedList.

2. Menambahkan elemen menggunakan add().
3. Menggunakan remove(), peek(), dan size().

2.4.3 Contoh Kode

```

1 package Pekan4_2511533026;
2 import java.util.LinkedList;
3
4
5 public class QueueLinkedList_2511533026 {
6     public static void main (String[] args) {
7         Queue<Integer> q_3026 = new LinkedList<>();
8         //tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
9
10        for (int i = 0; i < 6; i++)
11            q_3026.add(i);
12        //Menampilkan isi antrian.
13        System.out.println("Elemen Antrian "+ q_3026);
14        //Untuk menghapus kepala antrian.
15        int hapus = q_3026.remove();
16        System.out.println("Hapus elemen + " + hapus);
17        System.out.println(q_3026);
18        //Untuk melihat antrian terdepan
19        int depan = q_3026.peek();
20        System.out.println("Kepala Antrian = " + depan);
21
22        int banyak = q_3026.size();
23        System.out.println("Size Antrian = " + banyak);
24    }
25 }

```

2.4.4 Hasil

```

Console X
<terminated> QueueLinkedList_2511533026 [Java Application] C:\Users\Lenovo\p2\pool\plug
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen + 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5

```

2.4.5 Analisis

Implementasi ini lebih fleksibel karena ukuran dapat bertambah secara otomatis, namun sedikit lebih lambat dibandingkan array karena overhead pointer.

2.5 Aplikasi Queue – Reverse Data (ReverseData)

2.5.1 Teori

Salah satu aplikasi Queue adalah membalik urutan data dengan bantuan struktur data lain (dalam kasus ini Stack). Prinsipnya adalah memindahkan data dari Queue ke Stack (membalik), kemudian mengembalikan ke Queue.

2.5.2 Langkah-langkah

1. Mengisi Queue dengan data.

2. Memindahkan semua elemen Queue ke Stack.
3. Memindahkan kembali dari Stack ke Queue (sehingga urutan terbalik).

2.5.3 Contoh Kode

```
1 package Pekan4_2511533026;
2
3 import java.util.LinkedList;
4
5 public class ReverseData_2511533026 {
6
7     public static void main (String[] args) {
8         Queue<Integer> q_3026 = new LinkedList<Integer>();
9         q_3026.add(1);
10        q_3026.add(2);
11        q_3026.add(3); // [1, 2, 3]
12        System.out.println("sebelum reverse" + q_3026);
13        Stack<Integer> s = new Stack<Integer>();
14        while (!q_3026.isEmpty()) { // Q -> S
15            s.push(q_3026.remove());
16        }
17        while (!s.isEmpty()) { // S -> Q
18            q_3026.add(s.pop());
19        }
20        System.out.println("sesudah reverse = " + q_3026); // [3, 2, 1]
21    }
22 }
23 }
24 }
```

2.5.4 Hasil

```
<terminated> ReverseData_2511533026 [Java Applic
sebelum reverse[1, 2, 3]
sesudah reverse = [3, 2, 1]
```

2.5.5 Analisis

Kode ini berhasil membalik urutan elemen dari [1,2,3] menjadi [3,2,1] dengan efisien menggunakan kombinasi Queue dan Stack

2.6 Tugas Pekan 2

2.5.1 Analisis Kode Program

Berdasarkan implementasi pada kode sumber, terdapat beberapa poin kunci dalam mekanismenya:

- Manajemen Indeks Statis: Program menggunakan array `String[]` queue dengan kapasitas tetap yang ditentukan saat inisialisasi. Variabel `front` selalu menunjuk ke kepala antrian (indeks 0), sedangkan `rear` menunjuk ke elemen terakhir yang masuk.
- Mekanisme Pergeseran Data (*Shifting*): Hal yang paling krusial dalam kode ini adalah metode `dequeue()`. Berbeda dengan *circular queue*, program ini melakukan pergeseran elemen secara manual menggunakan perulangan `for` setiap kali satu data dihapus. Semua elemen di belakang akan maju satu posisi ke depan untuk mengisi kekosongan, kemudian indeks `rear` dikurangi (`rear--`).
- Algoritma Pembalikan (*Reverse*): Metode `reverse()` menerapkan algoritma *two-pointer*. Program menggunakan dua indeks, `i` (depan) dan `j` (belakang), yang bergerak saling mendekat ke tengah sambil menukar nilai elemen menggunakan variabel `temp`. Ini memungkinkan urutan pelayanan diubah secara total tanpa merusak struktur antrian.
- Validasi Kondisi: Program memiliki pengecekan `isFull()` untuk mencegah *overflow* saat menambah data dan `isEmpty()` untuk mencegah *underflow* saat menghapus atau menampilkan data.

2.5.2 Analisis Hasil Eksekusi

Dari hasil pengujian program melalui menu interaktif, dapat disimpulkan beberapa hal sebagai berikut:

- Prinsip FIFO (First-In, First-Out): Program secara akurat menjalankan prinsip antrian dasar. Pelanggan yang pertama kali dimasukkan melalui menu "Tambah Antrian" akan menjadi yang pertama kali dipanggil dan keluar melalui menu "Hapus Antrian".

- **Integritas Data:** Proses pergeseran data (shifting) pada saat dequeue memastikan bahwa antrian selalu rapat ke depan (indeks 0). Meskipun secara komputasi kurang efisien dibandingkan *circular queue* untuk data skala besar ($O(n)$), namun untuk skala kecil seperti simulasi loket ini, hal tersebut menjaga kemudahan dalam pembacaan indeks saat pemanggilan fungsi display().
- **Efektivitas Fungsi Reverse:** Saat fungsi pembalikan dipanggil, pelanggan yang tadinya berada di urutan paling belakang menjadi urutan pertama. Hal ini berguna dalam skenario simulasi tertentu di mana prioritas pelayanan perlu dibalik secara mendadak.
- **Interaktivitas User:** Penggunaan blok do-while dan switch-case pada metode main memberikan alur program yang dinamis, memungkinkan pengguna untuk terus mengelola antrian hingga memilih opsi keluar.

```

17     for (int i = 0; i < rear; i++) {
18         queue[i] = queue[i + 1];
19     }
20     rear--;
21 }
22
23 // Menampilkan antrian dari antrian[0:rear]
24 public void display() {
25     if (isEmpty()) {
26         System.out.println("Antrian kosong.");
27     } else {
28         System.out.println("Isi antrian:");
29         for (int i = front; i <= rear; i++) {
30             System.out.print((i - front + 1) + ". " + queue[i]);
31         }
32     }
33 }
34
35 // Membalik seluruh isi antrian[0:rear]
36 public void reverse() {
37     if (isEmpty()) {
38         System.out.println("Antrian kosong, tidak bisa reverse.");
39         return;
40     }
41     int i = front;
42     int j = rear;
43     while (i < j) {
44         int temp = queue[i]; // Simpan nilai awal i ke temp
45         queue[i] = queue[j];
46         queue[j] = temp;
47         i++;
48         j--;
49     }
50     System.out.println("Antrian berhasil dibalik.");
51 }
52
53 public static void main(String[] args) {
54     Scanner input = new Scanner(System.in);
55     // Inisialisasi objek loket dengan kapasitas belakang (maksud: 10)
56     AntrianLoket_2511533026 loket = new AntrianLoket_2511533026(10);
57     int pilihan;

```

```

58     do {
59         System.out.println("\n===== ANTRIAN LOKET =====");
60         System.out.println("1. Tambah Antrian");
61         System.out.println("2. Hapus Antrian");
62         System.out.println("3. Tampilkan Antrian");
63         System.out.println("4. Reverse");
64         System.out.println("5. Keluar");
65         System.out.print("Pilih menu: ");
66         pilihan = Input.nextInt(); // Memeriksa buffer
67         Input.nextLine();
68
69         switch (pilihan) {
70             case 1:
71                 System.out.print("Masukkan nama pelanggan: ");
72                 String nama = Input.nextLine();
73                 loket.enqueue(nama);
74                 break;
75             case 2:
76                 loket.dequeue();
77                 break;
78             case 3:
79                 loket.display();
80                 break;
81             case 4:
82                 loket.reverse();
83                 loket.display();
84                 break;
85             case 5:
86                 System.out.println("Program selesai.");
87                 break;
88             default:
89                 System.out.println("Pilihan tidak valid");
90         }
91     } while (pilihan != 5);
92     Input.close();

```

BAB III

KESIMPULAN

Praktikum pekan keempat ini memberikan pemahaman komprehensif mengenai struktur data Queue dan implementasinya dalam bahasa Java. Penulis berhasil mengimplementasikan Queue melalui dua pendekatan utama: berbasis array statis yang dikelola secara manual dan berbasis LinkedList dari Java Collections yang lebih dinamis. Melalui implementasi array, dipelajari pentingnya logika *circular queue* untuk mengoptimalkan penggunaan indeks memori, sementara melalui LinkedList, penulis memahami kemudahan integrasi dengan API Java seperti Iterator dan antarmuka Queue untuk kode yang lebih bersih dan efisien.

Selain itu, percobaan pembalikan data menggunakan Stack menegaskan bahwa struktur data tidak bekerja secara terisolasi, melainkan dapat dikombinasikan untuk memecahkan masalah logika yang lebih kompleks. Secara keseluruhan, prinsip FIFO pada Queue terbukti sangat efektif untuk manajemen tugas yang membutuhkan keadilan urutan proses. Penguasaan terhadap materi ini menjadi pondasi penting bagi penulis dalam merancang algoritma sistem terdistribusi maupun manajemen memori yang lebih canggih di masa mendatang.

DAFTAR PUSTAKA

1. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. Wiley.
2. Oracle. (2023). *The Java™ Tutorials – Lesson: Introduction to Collections*.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.